FULL STACK

LAB SHEET – 7

NAME – Vansh

CUID – CU24250033

SEC – B , ROLL – 64

```
CREATE DATABASE college_db;
USE college_db;   -- (PostgreSQL: \c college_db)
CREATE TABLE departments (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name       VARCHAR(50) NOT NULL,
    age        INT CHECK (age >= 17),
    city       VARCHAR(50),
    dept_id    INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    title     VARCHAR(80) NOT NULL,
    credits   INT DEFAULT 3,
    dept_id   INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
CREATE TABLE enrollments (
    student_id INT,
    course_id  INT,
    marks      INT,
    PRIMARY KEY (student_id, course_id),
```

```
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);
```

```
CREATE DATABASE IF NOT EXISTS college_db;
USE college_db;
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS enrollments;
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS courses;
DROP TABLE IF EXISTS departments;
DROP TABLE IF EXISTS faculty;
DROP VIEW IF EXISTS student_marks;
SET FOREIGN_KEY_CHECKS = 1;

CREATE TABLE departments (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE students (
   student_id INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   age INT CHECK (age >= 17),
   city VARCHAR(50),
   dept_id INT,
   FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE courses (
```

```
    course_id INT PRIMARY KEY,
    title VARCHAR(80) NOT NULL,
    credits INT DEFAULT 3,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    marks INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

INSERT INTO departments VALUES
(1,'CSE'),(2,'ECE'),(3,'Mechanical'),(4,'Civil');

INSERT INTO students VALUES
(101,'Aarav',20,'Roorkee',1),
(102,'Diya',21,'Dehradun',1),
(103,'Kabir',19,'Haridwar',2),
(104,'Meera',22,'Roorkee',1),
(105,'Rohan',20,'Delhi',3),
(106,'Sana',21,'Dehradun',2),
(107,'Vikram',23,'Jaipur',NULL),
(108,'Isha',19,'Roorkee',2);

INSERT INTO courses VALUES
```

```
(201,'Data Structures',4,1),
(202,'Operating Systems',4,1),
(203,'Digital Electronics',3,2),
(204,'Thermodynamics',3,3),
(205,'Machine Learning',4,1);

INSERT INTO enrollments VALUES
(101,201,88),(101,202,76),(101,205,91),
(102,201,67),(102,205,72),
(103,203,81),
(104,201,95),(104,202,89),
(105,204,58),
(106,203,74),
(108,203,66);

-- PART A: DDL
-- A1
CREATE TABLE faculty (
    faculty_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    salary DECIMAL(10,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
-- A2
ALTER TABLE faculty ADD phone VARCHAR(15);
-- A3
ALTER TABLE faculty RENAME COLUMN phone TO contact_no;
-- A4
```

```
ALTER TABLE faculty DROP COLUMN contact_no;
-- A5
DROP TABLE faculty;
-- DROP removes table structure and data; TRUNCATE removes all rows but keeps structure;
-- DELETE removes selected rows (or all rows) and keeps the table structure.

-- PART B: DML
-- B1
INSERT INTO students VALUES (109,'Neha',20,'Roorkee',1);
-- B2: omitted city defaults to NULL
INSERT INTO students (student_id,name,age,dept_id) VALUES (110,'Ankit',20,1);
SELECT * FROM students WHERE student_id=110;
-- B3
UPDATE students SET city='Roorkee' WHERE student_id=105;
-- B4
UPDATE enrollments SET marks=marks+5 WHERE course_id=203;
-- B5
DELETE FROM students WHERE student_id=109;
DELETE FROM students WHERE student_id=110;
-- B6: run separately to observe the CHECK constraint error:
-- INSERT INTO students VALUES (111,'Test',15,'Roorkee',1);

-- PART C: SELECT
-- C1
SELECT * FROM students;
-- C2
SELECT name,city FROM students;
-- C3
SELECT * FROM students WHERE city='Roorkee';
-- C4
```

```
SELECT * FROM students WHERE age BETWEEN 20 AND 22;
-- C5
SELECT * FROM students WHERE city IN ('Roorkee','Dehradun');
-- C6
SELECT * FROM students WHERE name LIKE 'A%';
-- C7
SELECT * FROM students WHERE dept_id IS NULL;
-- C8
SELECT DISTINCT city FROM students;
-- C9
SELECT * FROM students ORDER BY age DESC,name ASC;
-- C10
SELECT * FROM students ORDER BY age DESC LIMIT 3;
-- C11
SELECT name AS `Student Name`,age AS Age FROM students;

-- PART D: FUNCTIONS AND AGGREGATION
-- D1
SELECT COUNT(*) AS total_students FROM students;
-- D2
SELECT AVG(marks) AS average_marks,MIN(marks) AS minimum_marks,
    MAX(marks) AS maximum_marks FROM enrollments;
-- D3
SELECT city,COUNT(*) AS student_count FROM students GROUP BY city;
-- D4
SELECT c.title,AVG(e.marks) AS average_marks
FROM courses c LEFT JOIN enrollments e ON c.course_id=e.course_id
GROUP BY c.course_id,c.title;
-- D5
SELECT city,COUNT(*) AS student_count FROM students
```

```
GROUP BY city HAVING COUNT(*)>1;
-- D6
SELECT d.dept_name,COALESCE(SUM(c.credits),0) AS total_credits
FROM departments d LEFT JOIN courses c ON d.dept_id=c.dept_id
GROUP BY d.dept_id,d.dept_name;
-- D7
SELECT UPPER(name) AS uppercase_name,LENGTH(name) AS name_length FROM students;
-- D8: WHERE filters rows before grouping; HAVING filters groups after aggregation.

-- PART E: JOINS
-- E1
SELECT s.name,d.dept_name FROM students s
INNER JOIN departments d ON s.dept_id=d.dept_id;
-- E2: Vikram has no department, so dept_name is NULL.
SELECT s.name,d.dept_name FROM students s
LEFT JOIN departments d ON s.dept_id=d.dept_id;
-- E3: Civil has no students in the supplied data.
SELECT d.dept_name,s.name FROM students s
RIGHT JOIN departments d ON s.dept_id=d.dept_id;
-- E4
SELECT s.name,c.title,e.marks FROM students s
JOIN enrollments e ON s.student_id=e.student_id
JOIN courses c ON e.course_id=c.course_id;
-- E5
SELECT DISTINCT s.name FROM students s
JOIN enrollments e ON s.student_id=e.student_id
JOIN courses c ON e.course_id=c.course_id
WHERE c.title='Data Structures';
-- E6
SELECT s.name FROM students s LEFT JOIN enrollments e
```

```
ON s.student_id=e.student_id WHERE e.course_id IS NULL;
-- E7
SELECT c.title,COUNT(e.student_id) AS enrolled_students
FROM courses c LEFT JOIN enrollments e ON c.course_id=e.course_id
GROUP BY c.course_id,c.title;
-- E8
SELECT d.dept_name,COUNT(s.student_id) AS student_count,AVG(s.age) AS average_age
FROM departments d LEFT JOIN students s ON d.dept_id=s.dept_id
GROUP BY d.dept_id,d.dept_name;

-- PART F: SUBQUERIES
-- F1
SELECT * FROM students WHERE age>(SELECT AVG(age) FROM students);
-- F2
SELECT s.name,c.title,e.marks FROM enrollments e
JOIN students s ON e.student_id=s.student_id
JOIN courses c ON e.course_id=c.course_id
WHERE e.marks=(SELECT MAX(marks) FROM enrollments);
-- F3
SELECT * FROM courses c
WHERE EXISTS (SELECT 1 FROM enrollments e WHERE e.course_id=c.course_id);
-- F4
SELECT s.student_id,s.name,COUNT(*) AS course_count
FROM students s JOIN enrollments e ON s.student_id=e.student_id
GROUP BY s.student_id,s.name HAVING COUNT(*)>2;
-- F5 (ties are included)
SELECT d.dept_name,COUNT(s.student_id) AS student_count
FROM departments d LEFT JOIN students s ON d.dept_id=s.dept_id
GROUP BY d.dept_id,d.dept_name
HAVING COUNT(s.student_id)=(
```

```
  SELECT MAX(cnt) FROM (
    SELECT COUNT(*) AS cnt FROM students
    WHERE dept_id IS NOT NULL GROUP BY dept_id
  ) AS counts_by_department
);
-- F6
SELECT s.name,c.title,e.marks FROM enrollments e
JOIN students s ON s.student_id=e.student_id
JOIN courses c ON c.course_id=e.course_id
WHERE e.marks>(
  SELECT AVG(e2.marks) FROM enrollments e2 WHERE e2.course_id=e.course_id
);

-- PART G: INDEXES, VIEWS AND TRANSACTIONS
-- G1
CREATE INDEX idx_students_city ON students(city);
EXPLAIN SELECT * FROM students WHERE city='Roorkee';
-- G2
CREATE VIEW student_marks AS
SELECT s.name AS student_name,c.title AS course_title,e.marks
FROM students s JOIN enrollments e ON s.student_id=e.student_id
JOIN courses c ON e.course_id=c.course_id;
-- G3
SELECT * FROM student_marks WHERE marks>80;
-- G4
DROP VIEW student_marks;
DROP INDEX idx_students_city ON students;
-- G5: execute and inspect result, then rollback.
START TRANSACTION;
UPDATE enrollments SET marks=100 WHERE student_id=101 AND course_id=201;
```

```
ROLLBACK;
SELECT marks FROM enrollments WHERE student_id=101 AND course_id=201;
-- G6: execute separately; this commits the change permanently.
START TRANSACTION;
UPDATE enrollments SET marks=100 WHERE student_id=101 AND course_id=201;
COMMIT;
SELECT marks FROM enrollments WHERE student_id=101 AND course_id=201;

-- PART H: CHALLENGE PROBLEMS
-- H1
SELECT s.student_id,s.name,SUM(e.marks) AS total_marks,
      RANK() OVER (ORDER BY SUM(e.marks) DESC) AS student_rank
FROM students s JOIN enrollments e ON s.student_id=e.student_id
GROUP BY s.student_id,s.name;
-- H2
SELECT s.name,c.title,e.marks,
 CASE WHEN e.marks>=90 THEN 'A'
     WHEN e.marks>=75 THEN 'B'
     WHEN e.marks>=60 THEN 'C'
     ELSE 'D' END AS grade
FROM enrollments e JOIN students s ON s.student_id=e.student_id
JOIN courses c ON c.course_id=e.course_id;
-- H3
SELECT s1.name AS student1,s2.name AS student2,s1.city
FROM students s1 JOIN students s2
ON s1.city=s2.city AND s1.student_id<s2.student_id
WHERE s1.city IS NOT NULL;
-- H4
SELECT MAX(marks) AS second_highest_marks FROM enrollments
WHERE marks<(SELECT MAX(marks) FROM enrollments);
```